InboxOps

Complete Project Specification & Documentation

Windows desktop application • Hosted backend • Email-to-action automation

Version 1.0 | 21 October 2026 | Proposed baseline

Purpose

A development-ready baseline for converting business email enquiries into evidence-backed cases, assigned tasks, deadlines, follow-ups and auditable outcomes.

Document map

1. Executive Summary
2. Vision, Goals and Non-Goals
3. Users, Roles and Permissions
4. Functional Requirements
5. Workflow and State Model
6. Business Scenarios
7. User Interface Specification
8. Technical Architecture and Stack
9. Data Model
10. API Specification
11. AI and Document Processing
12. Security, Privacy and Threat Model
13. Reliability and Operations
14. Testing and Acceptance Criteria
15. Deployment and Release Management
16. Monetisation and Validation
17. Risks, Assumptions and Open Decisions
18. Repository Structure and Build Order
19. Glossary and Document Status

1. Executive Summary

InboxOps is a Windows desktop application backed by a secure hosted service. It helps small B2B sales and quotation teams turn business emails into trackable commitments. It identifies relevant enquiries, extracts structured information with source evidence, lets staff review and approve proposed records, assigns ownership, monitors deadlines, and records outcomes.

The product does not replace email, a CRM, an ERP, or a human sales representative. Its initial value is operational reliability: every relevant enquiry should have a clear owner, a next action, a deadline where appropriate, and an auditable outcome.

- Primary user: sales coordinator or quotation executive.
- Primary buyer: owner, sales manager, or operations manager at a small B2B business.
- Initial workflow: quotation requests and sales enquiries.
- Deployment: Electron desktop client plus cloud backend and background workers.
- AI policy: AI proposes; users approve important changes and all outbound actions in the MVP.
- Commercial objective: validate a paid workflow with pilot customers before expanding scope.

Product hypothesis: teams will pay if InboxOps measurably reduces missed enquiries, manual data entry, and time spent tracking follow-ups. This hypothesis must be validated through customer interviews and paid pilots.

2. Vision, Goals and Non-Goals

2.1 Vision

Make business commitments hidden in email visible, assignable, traceable, and easier to complete.

2.2 MVP goals

- Connect one email provider securely.
- Ingest and deduplicate messages from selected folders/labels.
- Classify likely quotation requests and sales enquiries.
- Extract key fields and attach source evidence.
- Allow review, correction, approval, and rejection.
- Create cases and tasks with owners, deadlines, and next actions.
- Track case state, related replies, reminders, and resolution.
- Provide dashboard, search, filters, audit history, and basic export.
- Provide a Windows installer and documented upgrade/uninstall path.

2.3 Non-goals

- Replacing a full CRM, ERP, invoicing system, or email client.
- Autonomously sending customer emails or making commercial commitments.
- Supporting every email provider at launch.
- Processing every attachment format or untrusted executable content.
- Building antivirus or enterprise security monitoring.
- Training a bespoke foundation model.
- Guaranteeing that AI understands every email correctly.

2.4 Product principles

- Evidence over assertion: extracted values link to source material.
- Human control: uncertain fields and external actions require review.
- Recoverability: retries must not duplicate business records.
- Least privilege: users and integrations receive only required access.
- Explainability: users can see why a case is overdue or a suggestion was made.
- Small scope first: one segment and one provider before expansion.

3. Users, Roles and Permissions

3.1 Personas

Persona	Needs	Success outcome
Sales/quotation executive	Review requests, prepare quotations, follow up	Fewer forgotten requests; less copying
Sales manager	See workload, overdue items and bottlenecks	Reliable team visibility and escalation
Workspace administrator	Manage members, integrations and rules	Safe access and predictable setup
Business owner	Understand performance and value	Evidence of time saved and missed work reduced

3.2 Roles

Role	MVP permissions
Owner	Workspace settings, integrations, users, all cases and reports
Admin	Manage users, workflow settings and integrations when granted
Manager	View team cases, assign/reassign, escalate, view team reports
Member	View permitted cases, edit assigned cases, create tasks, submit review decisions
Read-only	View explicitly permitted cases and reports; no edits

3.3 Authorization rules

- Every API request authenticates the user and verifies workspace membership.
- Authorization is enforced by the backend, never only by the desktop UI.
- Every workspace data query is scoped to the authenticated organization.
- Integration changes, member removal, exports and bulk edits are auditable.
- For MVP, workspace-wide case visibility is simpler; add team-level restrictions only when customer requirements justify them.

4. Functional Requirements

4.1 Email integration and ingestion

- Connect through OAuth; start with mock email fixtures, then one provider selected from pilot-customer demand.
- Allow selection of monitored folders/labels where supported.
- Persist provider message ID, thread ID, sender, recipients, timestamp, subject, safe body/reference and attachment metadata.
- Use provider notifications where available plus periodic reconciliation.
- Deduplicate by provider account plus provider message ID; process events idempotently.
- Show connection state, last successful sync, last error and reconnect action.
- Disconnecting stops future ingestion and revokes/deletes tokens where supported.

4.2 Classification and extraction

- Classify as quotation request, sales enquiry, reply/update, irrelevant, or needs review.
- Extract customer/company, contact, request summary, reference, items, quantities, units, explicit customer deadline, requested action and evidence.
- Represent unknown values as null/unknown, never guessed.
- Store model version, extraction timestamp, validation results and review outcome.
- Validate AI output against a schema and deterministic business rules.
- Treat email and attachment content as untrusted data, not instructions.

4.3 Review queue

- Show proposed classification and fields beside original email/source.
- Highlight missing, conflicting, ambiguous or low-evidence fields.
- Allow approve, edit-and-approve, reject and defer.
- Require human confirmation for ambiguous dates, quantities, identity and case matching.
- Record reviewer, time, original suggestion and corrected value.

4.4 Cases and tasks

- Create cases from approved requests or manual entry.
- Track status, priority, customer, owner, messages, deadlines and next action.
- Allow multiple messages and tasks per case.
- Keep customer deadline, internal task deadline and follow-up time separate.
- Support notes, assignment history, attachments and resolution reason.
- Reopen a resolved/closed case when a meaningful new reply arrives; preserve history.

4.5 Dashboard and reports

- Show items awaiting review, unassigned cases, due today, overdue cases, waiting-for-customer and recently resolved.
- Filter by owner, status, priority, customer, date range and case type.
- Search case metadata and approved extracted fields.
- Report first response time, time-to-quotation where measurable, follow-up coverage, overdue rate and outcomes.
- Exclude demo/test records from production reports.

4.6 Notifications

- Use configured business hours and workspace timezone.
- Schedule reminders from internal due dates or follow-up dates.
- Escalate only according to configured rules.
- Use idempotency keys to prevent duplicate reminders.
- Support quiet hours and notification preferences.
- Make reminder failures visible and retryable.

5. Workflow and State Model

5.1 Main workflow

1. Receive provider event or discover message during reconciliation.
2. Verify provider event and enqueue ingestion.
3. Fetch message details through provider API.
4. Normalize metadata/content; validate attachment type and size.
5. Persist message metadata and deduplicate.
6. Classify message and check for an existing case match.
7. Extract structured fields and evidence.
8. Validate schema, dates, quantities and required fields.
9. Place proposal in review queue when required.
10. User approves, corrects, rejects or defers.
11. Create/update case and tasks transactionally.
12. Schedule reminders and update dashboard.
13. Link later replies only when matching is sufficiently reliable.
14. Resolve/close with outcome and preserve audit history.

5.2 Case states

State	Meaning	Allowed transitions
NeedsReview	Proposal not accepted	Open, Rejected
Open	Accepted case not yet actively worked	InProgress, Closed
InProgress	Work underway	WaitingCustomer, Resolved
WaitingCustomer	Customer response/information expected	InProgress, Resolved
Resolved	Work appears complete; outcome recorded	Closed, Open (reopen)
Closed	Formally closed	Open (authorised reopen)
Rejected	Proposal rejected as irrelevant/duplicate/etc.	No normal transition; audit-only correction

Overdue is a derived flag, not a primary case state. A case can be InProgress and overdue simultaneously.

5.3 Deadline semantics

- customer_deadline: explicitly requested by the customer; retain source evidence.
- internal_due_at: deadline for the assigned employee's next action.
- follow_up_at: time to check status or contact customer.
- resolved_at: time a user resolves the case.
- Store timestamps in UTC and display in workspace timezone.
- Do not convert vague language into a hard customer deadline. Label suggested internal dates as suggestions.

5.4 Case matching

- First preference: provider thread/conversation ID and connected mailbox identity.
- Use In-Reply-To/References headers where available.
- Secondary signals: normalized subject, participants, customer, reference number and time window.
- Never merge cases based only on subject similarity.
- Low-confidence matches go to manual review.
- Record why a message was linked and audit authorised unlinking.

6. Business Scenarios

A. New quotation request

1. Buyer requests 500 components and asks for a quotation by a specific date.
2. InboxOps extracts buyer, product, quantity, reference and deadline with evidence.
3. Employee confirms fields and assigns owner.
4. Case enters action queue with next action and due date.

Expected result: one case, a traceable source email, and an accountable owner. Edge cases: missing quantity, ambiguous date, unsupported attachment, no explicit deadline.

B. Forgotten enquiry

1. Scheduler checks active case deadlines.
2. Case is marked overdue without overwriting the original deadline.
3. Owner sees it in overdue queue.
4. Configured rule may notify manager.

Expected result: delay is visible and action is clear. Test owner leave, reassignment, changed deadline and duplicate escalations.

C. Customer replies in same thread

1. Match via provider thread ID and message headers.
2. Link reply to existing case.
3. Highlight changed quantities/deadlines for review.
4. Recalculate next action only after an authorised decision.

Expected result: updated request stays connected to history. Forwarded messages and ambiguous new threads require cautious matching.

D. Attachment contains the request

1. Validate file type/size and scan where supported.
2. Extract text from supported PDFs/spreadsheets; use OCR fallback where appropriate.
3. Link values to source page/cell when possible.
4. Flag unreadable or uncertain values for manual review.

Expected result: structured case with secure reference to source attachment. Test corrupted, encrypted, oversized and malicious files.

E. Two employees claim the same case

1. Both clients display current owner/state.
2. Backend validates assignment when saved.
3. One assignment succeeds; the other receives a conflict and latest state.
4. Record the change in audit log.

Expected result: one authoritative state without silent overwrite.

Additional mandatory scenarios

- Message moved/deleted in provider.
- Provider outage, revoked token, missed notification or rate limit.
- Duplicate and out-of-order events.
- Employee access removed during active session.
- Deadline crossing timezone or weekend.
- AI returns malformed or contradictory fields.
- Email includes prompt-injection instructions.
- Closed case receives a meaningful reply.
- Customer requests deletion of retained personal data.

7. User Interface Specification

7.1 Navigation

- Overview: summary cards, action list, overdue items and sync health.
- Action Queue: review, unassigned, due today, overdue, waiting on customer.
- Cases: searchable list and saved filters.
- Review Inbox: AI proposals and source evidence.
- Reports: operational metrics and export.
- Team: members, assignments and workload.
- Integrations: connected accounts, sync status and permissions.
- Settings: timezone, business hours, reminders, workflow rules and retention.

7.2 Case list columns

Column	Behaviour
Case ID	Stable human-readable identifier
Customer	Resolved customer/contact or Unknown
Subject/request	Short editable summary
Status	Current state
Owner	Assigned user or Unassigned
Next action	Most relevant active task
Due	Applicable deadline and overdue indicator
Updated	Last meaningful activity

7.3 Case detail

- Summary header: case ID, status, priority, customer, owner and deadline.
- Timeline: linked emails, notes, assignments, state changes and approvals.
- Extracted fields: value, source, review status and edit history.
- Tasks: title, assignee, due date, status and completion evidence.
- Attachments: secure references with access checks.
- Actions: assign, edit, add task, mark waiting, resolve, reopen and close.
- Every action has success/error feedback; no destructive silent changes.

7.4 Accessibility

- Keyboard-accessible navigation and forms.
- Visible focus states and readable contrast.
- Never rely on colour alone for status.
- Support loading, empty, offline, permission-denied and error states.
- Confirm bulk/destructive operations.
- Make dates, timezone, AI suggestions and source evidence unambiguous.

8. Technical Architecture and Stack

8.1 Components

- Electron main process: window lifecycle, secure OS integration, updater and controlled IPC.
- Renderer: React UI; no direct access to secrets or arbitrary Node APIs.
- Backend API: authentication, authorization, business rules and queries.
- Email connector: OAuth, message fetch, notification verification and reconciliation.
- Queue/workers: ingestion, parsing, AI extraction, reminders and reports.
- PostgreSQL: authoritative multi-user records.
- Private object storage if attachments must be retained.
- AI provider adapter: structured extraction and redaction policy.
- Observability: logs, metrics, alerts and error reporting.

8.2 Recommended stack

Layer	Recommendation	Notes
Desktop	Electron	Windows package; secure preload/IPC
Frontend	React + TypeScript + Vite	UI and reusable components
Styling	Tailwind CSS + accessible component library	Consistent keyboard-usable UI
API	Node.js + TypeScript + Fastify OR NestJS	Choose one framework
Database	PostgreSQL	Transactions and multi-tenant data
ORM	Prisma OR Drizzle	Choose one
Queue	Redis + BullMQ	Retries and scheduled work; can defer in mock prototype
Email	Gmail API OR Microsoft Graph	Choose one for MVP
AI	LLM structured output + validation	Evidence and review required
Documents	PDF/spreadsheet parsers + OCR fallback	Isolated processing
Testing	Vitest + API integration + Playwright	Unit, integration, end-to-end
Distribution	Windows installer + signing	Test upgrade/uninstall

8.3 Deployment shape

Production: desktop client → HTTPS API → PostgreSQL/queue → workers → provider APIs/AI service. The desktop client is not the authoritative scheduler. Hosted workers continue processing when user computers are off. Use mock email data during early development.

8.4 Architecture constraints

- Use a modular monolith initially; avoid premature microservices.
- Keep provider adapters separate from business logic.
- Background jobs must be idempotent and retry-safe.
- Webhook handlers enqueue work instead of performing long AI processing inline.
- Version API contracts and shared validation schemas.

9. Data Model

Entity	Purpose / key fields
Organization	id, name, timezone, plan/status, retention policy
User	id, organization_id, email, display_name, role, status
EmailConnection	id, organization_id, provider, account_email, scopes, sync cursor, status
EmailMessage	id, connection_id, provider_message_id, thread_id, headers, subject, received_at, body reference
Customer	id, organization_id, name, domains, contact details
Case	id, organization_id, customer_id, type, status, priority, owner_id, deadlines
CaseMessage	case_id, email_message_id, link_reason, linked_by, linked_at
ExtractedField	case/message ID, field_name, value, source reference, confidence, review status
Task	id, case_id, title, assignee_id, due_at, status, completion evidence
Attachment	id, email_message_id, storage key/reference, MIME type, size, scan status
WorkflowRule	id, organization_id, trigger, conditions, action, enabled
Notification	id, organization_id, recipient_id, event key, channel, status
AuditEvent	id, organization_id, actor_id, entity type/ID, action, timestamp, metadata
ProcessingJob	id, job type, idempotency key, status, attempts, next retry, error code

9.1 Relationships and integrity

An Organization has Users, EmailConnections, Cases, WorkflowRules and AuditEvents. An EmailConnection receives many EmailMessages. A Case links to messages through CaseMessage and has Tasks and ExtractedFields. Use an explicit linking record so matching rationale remains auditable.

- Use UUIDs or another stable non-guessable ID.
- Unique constraint on provider message ID within email connection.
- Index organization_id, status, owner_id, due timestamps and thread IDs.
- Use UTC timestamps; store workspace timezone separately.
- Use transactions for approval, case creation, assignment and state changes.
- Use version fields/optimistic concurrency for simultaneous edits.
- Scope every query by organization_id; consider row-level security as defence in depth.
- Document retention for message content, attachments, extracted fields and logs.

10. API Specification

10.1 Conventions

- Base path /api/v1; JSON request/response; ISO 8601 UTC timestamps.
- Authenticate every user route and verify workspace/role server-side.
- Use pagination, stable error codes and idempotency keys for retried actions.
- Use version/If-Match checks for concurrent updates.

Method / path	Purpose
GET /api/v1/me	Current user and workspace
GET /api/v1/cases	List/filter cases
POST /api/v1/cases	Create case manually
GET /api/v1/cases/{id}	Case detail and timeline
PATCH /api/v1/cases/{id}	Update permitted fields

Method / path	Purpose
POST /api/v1/cases/{id}/assign	Assign/reassign owner
POST /api/v1/cases/{id}/resolve	Resolve with outcome
POST /api/v1/cases/{id}/reopen	Reopen case
GET /api/v1/tasks	List tasks
PATCH /api/v1/tasks/{id}	Update task
GET /api/v1/review-queue	Pending proposals
POST /api/v1/review-queue/{id}/approve	Approve proposal
POST /api/v1/review-queue/{id}/reject	Reject proposal
GET /api/v1/integrations	Integration status
POST /api/v1/integrations/{id}/sync	Queue sync
GET /api/v1/reports/summary	Operational summary
POST /api/v1/webhooks/{provider}	Verified provider notification endpoint

10.2 Error format


```
{"error":{"code":"CASE_VERSION_CONFLICT","message":"This case changed since you opened it.","request_id":"req_example","details":{}}}
```


10.3 API security

- Validate input schemas and body sizes.
- Rate-limit authentication, exports and expensive AI operations.
- Never trust organization_id or role supplied by the client.
- Redact tokens and email bodies from logs.
- Verify provider-specific webhook credentials/signatures.
- Avoid detailed authentication errors that reveal account existence.

11. AI and Document Processing

11.1 Processing pipeline

1. Normalize headers and extract safe text.
2. Classify message type.
3. Decide whether likely new request, reply/update, irrelevant or ambiguous.
4. Extract allowed structured fields.
5. Attach source evidence to each field.
6. Validate types, dates, units and required fields in ordinary code.
7. Apply case matching policy and flag ambiguity.
8. Create review proposal; do not perform high-impact external actions.

11.2 Example output schema


```
{"message_type":"quotation_request","customer_name":"ABC Components","reference_number":"RFQ-1042","requested_items":[{"description":"Industrial c
```


Illustrative data only. Production schema must allow null/unknown fields, preserve original phrases/timezone where needed, and identify source message/document/page/cell.

11.3 Quality and safety rules

- Do not rely only on model-reported confidence; combine evidence, schema validity and rule checks.
- Ambiguous/high-risk fields require review regardless of numeric confidence.
- Track field-level accuracy on a labelled evaluation set.
- Record model and prompt/schema version for debugging.
- Do not train on customer content without explicit authorisation.
- Email and attachment text are data, never privileged instructions.
- AI tools must not have arbitrary database or email-sending capabilities.

11.4 Attachment processing

- Allow-list supported formats and enforce size limits.
- Scan files and store outside executable directories.
- Parse in isolated workers with time/resource limits.
- Never execute macros, scripts or embedded files.
- Use OCR only when needed and label uncertainty.
- Delete temporary copies according to retention policy.

12. Security, Privacy and Threat Model

12.1 Security baseline

- OAuth with least-privilege scopes and secure refresh-token handling.
- TLS in transit; encryption at rest for database, backups and stored attachments.
- Secrets in managed secret storage, not source control or renderer code.
- Electron: context isolation on, Node integration off in renderer, restrictive CSP, validated IPC.
- Tenant authorization tests and audit logging.
- Dependency/secret scanning, patching and signed release artifacts.
- Backup/restore drills and incident response documentation.

12.2 Privacy baseline

- Disclose mailbox scope, processing, AI use, retention and deletion at onboarding.
- Collect only messages needed for selected workflow.
- Provide disconnect, export and deletion procedures.
- Define retention for content, attachment copies, extracted fields, logs and backups.
- Minimise/redact data before external AI processing where feasible.
- Review applicable Indian data-protection law, provider terms and customer contracts with qualified counsel.

Threat	Control
Cross-tenant leakage	Tenant-scoped queries, backend authorization, negative tests, optional RLS
Token theft	Secure backend storage, least privilege, revocation and no token logging
Prompt injection	Treat content as untrusted; no autonomous privileged tools
Malicious attachment	Allow-list, scanning, isolated parsing, limits
Forged/replayed webhook	Provider verification, idempotency and replay-safe processing
Duplicate/out-of-order events	Idempotent jobs, unique constraints, reconciliation
Accidental outbound email	No auto-send in MVP; explicit confirmation and audit
Data loss	Encrypted backups and restore tests

13. Reliability and Operations

13.1 Background jobs

- Each job has an idempotency key, status, attempts, retry time and safe error code.
- Retry transient failures with exponential backoff and jitter.
- Do not retry permanent validation failures indefinitely.
- Provide failed-job/dead-letter visibility.
- Use reconciliation to recover missed provider events.
- Make handlers safe when executed more than once.

13.2 Monitoring

- Provider sync lag and last successful sync.
- Queue depth, job age, failure rate and retry count.
- AI validation error and review rejection rates.

- Reminder delivery success/failure.
- API latency/error rate and database saturation.
- Desktop crash and updater failure rate.

Measure	Initial goal
Dashboard latency	p95 under 2 seconds for typical cached view
Availability	Aim for 99.5% monthly after pilot; publish only when measured
Duplicate handling	Repeated event must not create duplicate case/message
Recovery	Transient retries and periodic reconciliation
Auditability	Important state changes record actor and timestamp
Data portability	Export cases/tasks and key history in documented format

These are proposed engineering goals, not guarantees. Measure them during the pilot.

14. Testing and Acceptance Criteria

14.1 Test layers

- Unit: date/deadline calculations, state transitions, permissions, schema validation and idempotency.
- Integration: database transactions, provider adapters, OAuth refresh, queue retries and webhook verification.
- End-to-end: connect mailbox, review proposal, create case, assign task, receive reply, resolve and export.
- Security: cross-tenant access, role escalation, malicious content, token leakage and webhook replay.
- Usability: keyboard navigation, empty states, correction flow and evidence clarity.
- Recovery: provider outage, worker crash, database restore, expired token and missed notification.

14.2 Release acceptance checklist

- Connect/disconnect provider and explain permissions.
- Deduplicate messages and preserve source links.
- Show evidence for extracted values and allow correction.
- Persist cases/tasks/deadlines and sync between users.
- Link replies only when matching is reliable.
- Respect timezone and business settings for overdue rules.
- Prevent silent concurrent assignment overwrites.
- Prevent cross-workspace access.
- Stop sync after access revocation and display warning.
- Handle AI errors, malformed attachments and provider outages safely.
- Test export, retention and deletion behaviour.
- Test Windows install, upgrade, rollback and uninstall.

15. Deployment and Release Management

15.1 Environments

- Local development: mock provider, synthetic seed data, local/development database.
- Staging: separate credentials/database, test mailbox and non-production AI configuration.
- Production: managed PostgreSQL, private object storage if used, queue, workers, monitoring and backups.

15.2 Release process

1. Run format, lint, type checking and unit tests.
2. Run integration and security regression tests.
3. Build installer from clean CI environment.
4. Sign artifacts when distributing publicly.

5. Deploy backward-compatible database migrations.
6. Release to pilot group and monitor errors.
7. Document rollback for backend and desktop versions.

15.3 Recovery

- Define recovery point objective and recovery time objective with pilot customers.
- Encrypt backups and restrict access.
- Test restoration regularly.
- Restore database and attachment storage consistently.
- Align retention/deletion policy with backup expiry.

16. Monetisation and Validation

Plan hypothesis	Illustrative price	Possible scope
Solo	■799/month	One user, limited processed messages
Team	■2,499/month	Small team, shared queue and assignments
Business	■4,999+/month	Higher volume, advanced roles/integrations

Prices are hypotheses, not established market rates. Define user limits, message/AI volume, support, integrations and tax treatment before publishing.

16.1 Value metrics

- Share of relevant enquiries with owner and next action.
- Median time from enquiry to first human response.
- Share of cases overdue by internal deadline.
- Manual correction rate for extracted fields.
- Minutes spent per enquiry before/after adoption.
- Weekly active users and repeated action-queue usage.
- Pilot conversion, paid retention, churn and support burden.

16.2 Validation plan

1. Interview 10–15 target businesses and observe their current process.
2. Prototype with synthetic or authorised anonymised emails.
3. Run a pilot with baseline measurements.
4. Ask for payment after demonstrating a real workflow outcome.
5. Build advanced integrations only after multiple customers request them and core usage retains.

17. Risks, Assumptions and Open Decisions

Risk	Mitigation
Existing competitors	Focus on commitment tracking for a narrow workflow
Incorrect extraction	Evidence, validation, review and accuracy measurement
Low willingness to pay	Customer discovery and paid pilots
Provider complexity	Start with one provider and isolate adapters
Sensitive email data	Least privilege, minimisation, review and clear retention
Scope creep	Freeze MVP; defer autonomous agents and broad vertical support
Background failures	Queue, retries, reconciliation and alerts

17.1 Assumptions to validate

- Target teams receive enough email enquiries for missed follow-ups to be costly.
- Customers will permit a third party to process selected mailbox data.
- Human-reviewed extraction saves enough time to justify subscription.

- Teams will use an action queue instead of relying entirely on email/spreadsheets.
- One provider covers a meaningful share of pilot customers.

17.2 Decisions still open

- First provider: Gmail/Google Workspace versus Microsoft 365; decide from pilot evidence.
- Attachment storage: provider reference versus encrypted copy with explicit retention.
- Hosting region based on customer, cost, privacy and contractual requirements.
- AI vendor/model based on quality, terms, latency and per-message cost.
- Whether restricted team-level case access is required in MVP.
- Final subscription limits/prices based on paid-pilot evidence.

18. Repository Structure and Build Order

```
inboxops/\n  apps/\n    desktop/         # Electron main, preload, React renderer\n    api/             # Authenticated REST API\n  workers/\n
```

18.1 Build order

1. Create repository, checks and synthetic seed data.
2. Build UI with mock email data and realistic review/case workflows.
3. Implement database models, authentication and tenant authorization.
4. Implement cases, tasks, assignments, deadlines and audit events.
5. Implement one provider integration.
6. Add ingestion queue, deduplication, retry and reconciliation.
7. Add AI extraction, evidence references and human review.
8. Add reminders and basic reports.
9. Add safe attachment parsing.
10. Complete security, recovery, packaging and pilot acceptance tests.

Definition of done: a feature is complete only when requirements, UI states, backend authorization, persistence, audit implications, error handling, automated tests and user documentation are complete. A mock-only screen is a prototype, not a production feature.

19. Glossary and Document Status

Term	Definition
Case	Business enquiry/commitment linking messages, tasks and outcomes
Task	Discrete action assigned to a user with optional due date/evidence
Commitment	Promise, request or expected action tracked to outcome
Review proposal	AI interpretation awaiting human decision
Idempotency	Repeated processing without duplicating business result
Reconciliation	Periodic provider check to recover missed events/state changes
Tenant/organization	Customer workspace whose data must be isolated
RPO	Maximum acceptable data loss measured in time
RTO	Target time to restore service after outage

This is a proposed baseline specification. It does not claim that software has been implemented, security-tested, legally reviewed, or validated with paying customers. Update it as implementation choices and pilot evidence become available.